

Leiden University - Deep Learning Task 2 (Group 57)

Manuel Pérez Belizón
4416880

Rejdi Danaj
3988651

Valentin Kodderitzsch
3895157

December 2024

All authors contributed equally to this group project.

1 GAN & VAE (Task 1)

The objective of this coursework is to explore generative models for image-related tasks by understanding the principles behind simple generative architectures. We worked with a [provided Jupyter notebook](#) that contains skeleton code to build and train Variational Autoencoders (VAE) and Generative Adversarial Networks (GAN). The main assignment involved selecting a custom image dataset, preprocessing it, retraining the models, and generating novel content using the trained networks.

1.1 Selected dataset

The dataset we chose is the animal faces dataset from Kaggle which contains 3 types of animals: cats, dogs, and wild animals such as lions and wolves. We chose the dog dataset for this assignment since it includes samples that are very closely related. Figure 1 shows that most dogs directly look at the camera and only their faces are shown. This reduces the number of distinct image features to the dog's nose, ear, eyes, mouth, and fur color. We believe that this is an interesting dataset as it is based on real-life images, which are more complex to replicate than artificial images like cartoon dogs for instance. The cats' dataset would have been a viable alternative as most samples are also closely related. However, the wild animals dataset would likely be challenging as the samples are more diverse.

The dog face dataset contains 4739 images in the training set and 500 in the validation set. Each image is an RGB image of size 512x512 pixels. As a first pre-processing step we added the validation set images to the train set. Our task is to generate new images so we do not need a validation set anymore to evaluate them formally. Instead, we will perform a visual inspection. The bigger the training set the better the generative capabilities should be. Then, we changed the image size to 96x96x3, which is small enough for the models to train fast, but large enough to capture important image features. As a second pre-processing step we converted the images from jpeg to .npy format so that the models which use the Keras library can work with them. Depending on the model's activation function we rescaled the pixel intensity values to a $[0, 1]$ range (for ReLu) or a $[-1, 1]$ range (for tanh) as a final step.



Figure 1: Training set images (random selection)

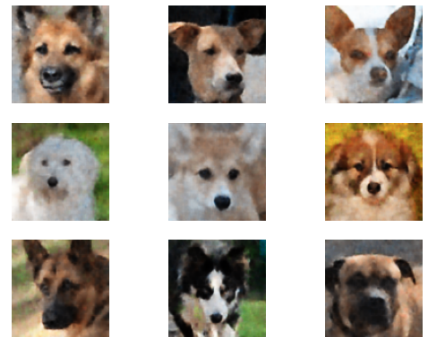


Figure 2: CAE re-constructed images

1.2 CAE training

For this assignment, we first experimented with Convolutional Autoencoders (CAEs) to understand how the basic encoder-decoder architecture works, as they serve as building blocks for both VAEs and GANs. A CAE is an encoder-decoder neural network (NN) architecture designed for feature extraction and data reconstruction. The encoder compresses the original dog image X into a lower-dimensional latent representation Z , capturing key features through convolutional layers. This is conceptually similar to classic dimensionality reduction techniques like PCA, though autoencoders can model complex, non-linear relationships. The decoder then reconstructs the input image from Z , learning to generate a version of X that closely resembles the original. Unlike generative models like VAEs, standard CAEs do not have explicit generative capabilities.

Figure 2 shows reconstructed images from Figure 1. Since our encoder-decoder network uses a stride size of 2, the input image size is halved per layer in the encoder and doubled in the decoder. With an input image size of 96x96 pixels, the feature map size becomes $96/(2^n) \times 96/(2^n)$ after n layers. The smallest possible feature map size is 1x1, limiting the network to a maximum of 6 layers. We found that a CAE with 2 hidden layers, a latent dimension of 512, and 128 convolutional filters per layer produced the best reconstructions. We used a batch size of 2, resulting in about 2600 images per epoch, which helped stabilize training results. All other NN parameters like tuning rate, activation functions, etc. were left like in the original skeleton code notebook.

The reconstructed images closely resemble the originals, suggesting that our parameter choices were valid. As expected, increasing the latent space and the number of convolutional filters improved the reconstructions by capturing more information. Surprisingly, using 2 hidden layers outperformed 4 layers. In hindsight, fewer layers may better preserve details, as deeper architectures can cause excessive dimensionality reduction, potentially losing important features.

1.3 VAE training

We initially trained the VAE using insights from the CAE training. A large latent space, many convolutional filters, and few hidden layers, while keeping other architectural choices identical to the original notebook. However, this approach failed. The VAE produced completely gray images with no recognizable features. We suspect this was due to the latent space being too large and allowing for redundant encoding. As such, the decoder cannot pick up any useful latent information and returns a noisy, non-informative image.

Using the TensorFlow VAE tutorial [2] as a guide, we refined our parameters to a latent space of size 4, 32 convolutional filters, and two hidden layers. We matched the effective training size per epoch to the original notebook by setting the batch size to 2. During training, we sampled nine random vectors from the latent space for visualization and displayed the corresponding decoded images after each iteration. The quality of these reconstructions improved over time.

Our final result, shown in Figure 3, showcases nine recognizable dog faces. However, the faces lack variety and appear blurred, as if viewed through an opaque window. After reading Chapter 17 of the Understand Deep Learning [1] textbook, we learned that this is expected. As the textbook notes, "unfortunately, samples from vanilla VAEs are generally low-quality" (page 340). Therefore, we believe that our final results are valid.

1.4 GAN training

For the GAN training process, we decided to use the VAE training insights instead of the CAE insights, as VAEs and GANs are both generative models. Therefore, we first used a small latent space, few convolutional filters and few hidden layers. We kept the batch size at 2 and pre-processed the images to be in the $[-1, 1]$ range, as specified in the original notebook. However, the CAE insights did not translate to the GAN training process, as the generated images were very noisy, colorful and showed no distinct facial features. This was an unexpected result. Our explanation is that different generative architectures prefer different ranges for the latent space and number of convolutional filters. We believe this is analogous to classic regression, where different regression models from the same family, e.g. stochastic boosting and XGboost, might prefer different ranges despite sharing the same hyperparameter. As such, our suspicion is that the VAE's preferences for a smaller latent space and fewer convolutional filters is simply too small for the GAN to extract any meaningful features for the decoding process.

Guided by a TensorFlow GAN tutorial [3], we adjusted the parameters. The best-performing model used a latent space of size 150, 350 convolutional filters, two hidden layers and a batch size of 2. All other parameters were the same as in the original notebook. For evaluation, we sampled nine random vectors from the latent space and visualized the corresponding generated images after each training iteration.

Our final result is shown in Figure 4. Compared to the VAE generated images the GAN images are much more colorful and diverse. However, about half of them are so "diverse" that distinct facial features like eyes or ears are obscured by "colorful noise". This was an expected result though as the Understanding Deep Learning textbook [1] states in chapter 15 that "GANs are notoriously difficult to train" (page 279). Our GAN generated images look similar to the example GAN generated images from the textbook, so we believe that our results are valid.

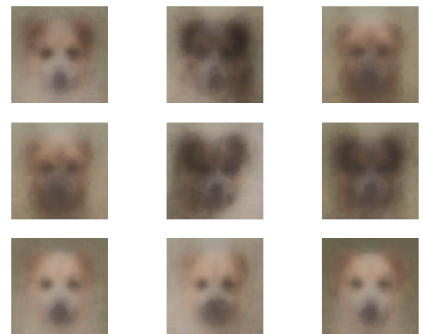


Figure 3: VAE generated images



Figure 4: GAN generated images

1.5 Interpolation

As part of this assignment, we were explicitly asked to create a visualization of a linear interpolation between two randomly sampled vectors from the latent space. Generative models can take such vectors as input to the decoder (in VAEs) or generator (in GANs) to produce novel images in the original feature space. These capabilities are demonstrated in Figures 3 and 4, where none of the dogs shown are reconstructions of specific original images. Instead, they are novel, artificially generated representations based on common facial features learned from the original, real-life dog images in the training dataset.

Mathematically, linear interpolation is straightforward. Consider two scalar values, $x = 5$ and $y = 9$. Moving from x to y involves adding 4 to x , which is equivalent to $x + (y - x)$. To take only half a step between x and y , we compute $x + 0.5(y - x)$. Generalizing this, we use the formula $x + \alpha(y - x)$, where $\alpha \in [0, 1]$ controls the interpolation between x and y . The same principle extends to vectors. For this task, we chose $\alpha \in \{0, 0.25, 0.5, 0.75, 1\}$ to create evenly spaced transitions. Figure 5a shows the interpolation results from the VAE, while Figure 5b shows those from the GAN. Given that we believe our VAE and GAN training results are valid, we also consider our interpolation results to be valid. The visualizations behave as expected: at $\alpha = 0.5$ the generated image appears to be an approximate 50:50 blend of the images produced from vectors x and y .

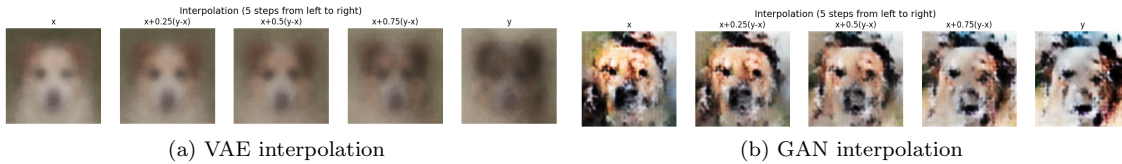


Figure 5: Linear interpolation from random sample x to y , with step size $\alpha(y - x)$

1.6 VAE working principles

A VAE is a probabilistic extension of the CAE, incorporating principles from Bayesian statistics. Like a CAE, a VAE compresses high-dimensional input data into a lower-dimensional latent representation using neural network architectures such as convolutional layers. However, unlike a CAE, both the encoder and decoder in a VAE are designed as probabilistic models. Therefore, it is possible to draw samples from a latent space, which can then be decoded.

Before exploring VAEs in depth, please consider reading this [mini-tutorial](#) I wrote, which explains Bayesian statistics, likelihood functions, and parameter estimation. In the deep learning literature (such as our textbook [1], terms like "likelihood function" and "probability distribution" are often used interchangeably, which is technically wrong and cause confusion.

$$p(z|x) = \frac{p(x|z)p(z)}{\int p(x|z)p(z)dz} = \frac{N(x; \mu = f(z, \phi), \sigma = f(z, \phi)) \times N(0, 1)}{p(x)} \approx q(z|x) = N(\mu = g(x, \theta), \sigma = g(x, \theta)) \quad (1)$$

Unlike a traditional Bayesian workflow where you try to maximize the model evidence by changing the prior hyper-parameters, VAEs fix the prior $p(z)$ to be a standard normal $N(0, 1)$. The likelihood function $p(x|z)$ which is usually not a learnable function in a traditional Bayesian workflow, now hosts the decoder $f(z, \phi)$ with learnable weights ϕ , to estimate the likelihood parameters μ, σ . The posterior $p(z|x)$ is computationally intractable, like in most Bayesian settings, so it is approximated via $q(z|x)$ using variational inference, which is a common technique in Bayesian statistics. However, unlike a traditional Bayesian workflow where the posterior hyper-parameters are fixed post (conjugate) updating, the approximated posterior $q(z|x)$ hosts the encoder. The encoder $g(x, \theta)$ with learnable weights θ estimates the posterior parameters μ, σ . Like in a traditional Bayesian workflow, variational inference is implemented by maximizing the ELBO. However, due to the ELBO function consisting of the proposed posterior $q(z|x)$ (hosts encoder), the likelihood function $p(x|z)$ (hosts decoder) and the prior $p(z)$, the VAE implementation of maximizing the ELBO **simultaneously** minimizes the reconstruction error and the KL score between the proposed and the true posterior. Additionally, the ELBO is a differentiable loss function which is required to train a deep learning model. Finally, the re-parametrization trick is used to allow for stochastic gradient descent instead of classic gradient descent which cannot escape local minima.

1.7 GAN working principles

Unlike CAEs and VAEs, GANs do not have the typical encoder-decoder architecture that compresses variables into latent space. Instead, GANs host 2 separate neural networks that compete with each other in a zero-sum game, so one

network's win is the other network's loss. The 2 networks are called generator and discriminator. The generator creates novel, synthetic images, while the discriminator tries to distinguish between real and fake images. The generator's goal is to maximize the discriminator's error, while the discriminator's goal is to minimize its error.

The generator, similar to the decoder in VAEs, takes a latent vector as input and transforms it into high-dimensional data using convolutional layers. However, unlike VAEs the latent vector is sampled from "random noise" (e.g. a $N(0,1)$), whereas VAEs "learn" how to parameterize the latent space distribution. The discriminator is a binary classification model which receives both real data (from a dataset) and fake data (from the generator). Over the training iterations the discriminator becomes more powerful at classifying fake data, which in turn encourages the generator to create more realistic looking images.

The training process alternates between updating the discriminator and the generator. In each iteration, the discriminator is updated first to improve its distinguishing abilities. Once the discriminator is updated, the generator is trained to improve its ability to fool the discriminator into classifying fake images as real. This iterative process continues, with the goal of reaching an equilibrium where the generator produces high-quality artificial images that the discriminator cannot reliably distinguish from real ones. The GAN loss function combines the loss functions of the discriminator and the generator networks at the same time. Ideally, both generator and discriminator loss decrease at the same pace, resulting in stable image generation. Both network's loss functions are based on the binary cross-entropy.

1.8 VAE vs GAN

VAE and GANs are both generative models. However, they were designed based on different philosophies. VAEs have a Bayesian approach, which is reflected in its loss function, ie. the maximisation of the ELBO. GANs have a more practical, computer science based design philosophy, combining existing neural networks to create a new network for a specialized task. This philosophy is again reflected in the GANs loss function, which combines the generator's and the discriminator's loss functions. Due to the Bayesian model building approach, VAEs learn a smooth, probabilistic latent space via the encoder-decoder architecture, whereas GANs use a random latent space sampled from a fixed prior distribution. GANs are generally less stable to train due to the adversarial competition between the generator and discriminator, where imbalances between the two can lead to training difficulties. As a result of the variational approximation on the latent space, VAE-generated samples tend to be less diverse compared to GAN-generated outputs, which can produce highly varied data.

2 RNN & LSTMs (Task 2)

The objective of Task 2 is to develop and train 3 different Recurrent Neural Networks (RNNs) that can add or subtract two integers that are at most two digits long. In this assignment, three different models, text-to-text, image-to-text and text-to-image. The objective of these models is to correctly calculate the solution to arithmetic operation. Each model will have a different architecture to handle each input and output representation, text or image.

2.1 Data exploration

The dataset contains 20,000 samples. Each sample includes an **Input**, an addition or subtraction operation represented with exactly 5 characters, the sequence is padded with spaces if needed (e.g., "1+20 ") and a **Label**, the solution to the operations with exactly 3 characters, again the sequence is padded with spaces if needed (e.g., "21 ").

2.1.1 Operation classification

We can classify all those operations in 13 different operation types depending on: (i) Arithmetic type: Addition or subtraction; (ii) Digit configuration: 1 digit $\pm$ 1 digit, 1 digit $\pm$ 2 digit (equivalent to 2 digit $\pm$ 1 digit) and 2 digit $\pm$ 2 digit; and (iii) Carry: Whether the operation involves carry or no carry.

However, the class "1 digit - 1 digit with carry" is not possible giving us 11 valid different classes. Additionally, there are 2 more classes that appear in addition: (i) Double carry (e.g. 99 + 99) and (ii) First digit carry (e.g., 90+70). With this classification we can check the operation distribution of the input of our model as well as the mistakes distribution to analyze what type of operations are our models misclassifying the most.

2.1.2 Dataset split

We will try different train-test splits for training and evaluating our models. We are going to try four different train-test splits, 90,75,50 and 10%. We see from Figure 6 that none of the operation types are missing in any of the splits, but

some operation types might be underrepresented (e.g. only 8 1 digit $\pm$ 1 digit operations in the 10% split), which could lead the model to not properly learn these operations.

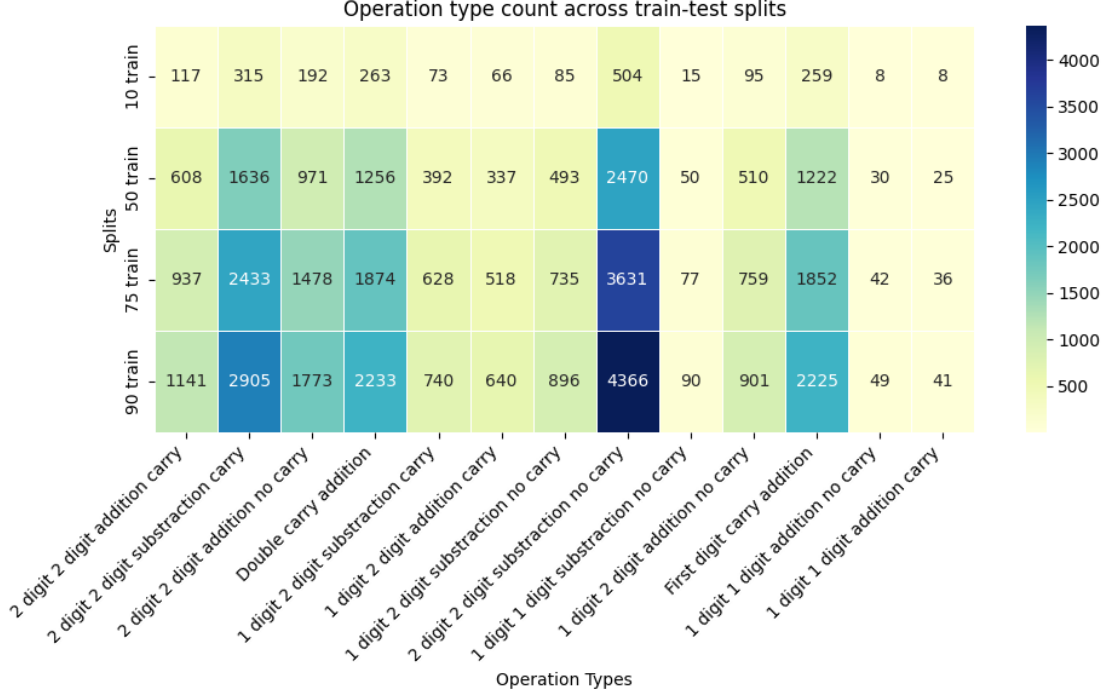


Figure 6: Operation type count across train-test splits

2.1.3 Data representation

This dataset has 2 different representations, the text representation where the input sequences are represented as strings of length 5 and the label sequences are represented as strings of length 3 and the image representation where the input sequences are represented as a stack of 5 28x28 images and the label sequences are represented as a stack of 3 28x28 images. Based on how the inputs and labels are represented, we are going to develop 3 different models.

2.2 Text-to-text RNN model

The first model is a text-to-text recurrent neural network where both the input and the output sequences are represented as text strings, for training this model it is necessary to encode the input and labels to one-hot format, so instead of actual 5 character strings, we will work with samples with (5,13) shape as each sample has 5 characters and there are 13 unique, the same format is applied to our labels where instead we will work with samples of (3,13) shape.

For this task the model itself was provided and so the main task is to evaluate the performance of the model for the different train-test splits.

2.2.1 Results

The losses and accuracies for each split are shown in Table 1, and we can see that the 10% and the 25% splits are the best in terms of performance. Moreover, Figure 7 shows the mistake distribution of operation type for the different splits.

These results suggest that the 90-10 and the 75-10 train-test splits are the most suitable for our models, therefore, we will only be using the 90-10 split for the rest of our models.

2.2.2 Discussions

As shown in the results, the splits with a smaller training set size, 50% and 10% training size struggle the most having 6183/10000 and 17088/18000 misspredictions, respectively. The results show that the bigger the train set is the better

Table 1: Text-to-text Splits Performance

Test Set	Split %	Loss	Accuracy
50		0.6027	0.7678
25		0.1980	0.9437
90		1.2366	0.5403
10		0.0730	0.9823

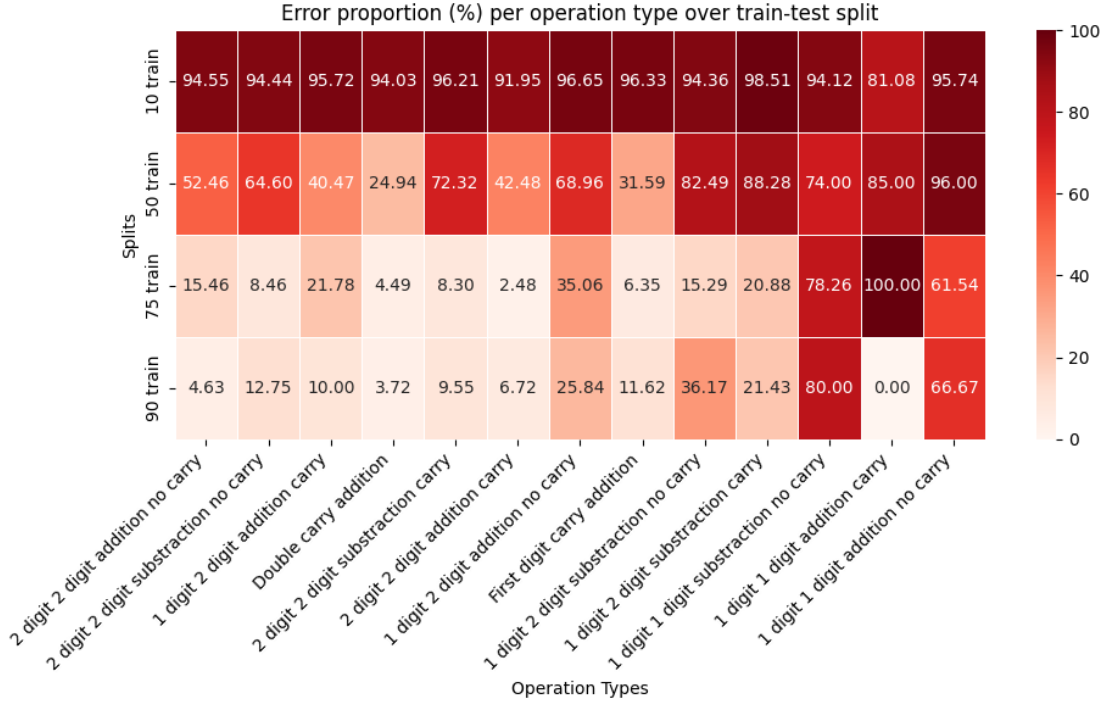


Figure 7: Error Proportion (%) per operation type over train-test splits

the model seems to learn the different operation types but the models keep struggling with underrepresented classes like 1 ± 1 no carry. This was an expected result as the models cannot really learn the general principles of those operations when the training set contains a very limited number of such operations.

2.3 Image-to-text RNN

The second model is an image-to-text RNN. In this model, the input sequences are represented as images and the output sequences are represented as text strings. Each input sequence consists of five 28x28-pixel images, each corresponding to a digit or the operator sign. The output sequences will have the same format as the output sequences of the previous text-to-text model. As shown in Table 1 we found that 90-10 train-test split gives the best accuracy and loss and therefore we will be using that split for the rest of the models.

2.3.1 Approach

We will be building, training, and evaluating four different models; the architectures of each model are shown in Table 2 below. In this task, the models will be built using an encoder-decoder architecture. This structure is explained in more detail in Section 2.4. Additionally, a convolutional LSTM layer will be used. These layers allow us to retain visual features sequentially by leveraging the memory logic of an LSTM. Convolutional LSTM layers are discussed further in the aforementioned Section 2.4.

The models trained in this task are separated into two groups: single-layer and multi-layer. The single-layer models are named "ConvLSTM" and "LSTM." Expectedly, the parameters of a fully connected architecture such as "LSTM" are much larger than those of its convolutional counterpart. As stated, the multi-layer models follow the same structure, with one additional layer added for both the encoder and decoder. We also observe the same parameter count trend in the multi-layer models.

Table 2: Model architectures

Model Name	Encoder Layers	Decoder Layers	nr.params
ConvLSTM (single)	ConvLSTM2D , GlobalAvgPooling2D, Flatten	RepeatVector,LSTM, Dense	482,061
LSTM (single)	Flatten,LSTM	RepeatVector,LSTM,Dense	1,594,637
ConvLSTM_multi	ConvLSTM2D, ConvLSTM2D, GlobalAvgPooling2D,Flatten	RepeatVector, LSTM,LSTM,Dense	1,270,853
LSTM_multi	Flatten, LSTM, LSTM	RepeatVector, LSTM, LSTM, Dense	3,142,093

2.3.2 Results

The results in Table 3 show how the different models performed, with the LSTM multi-layer model performing the best, achieving 77% accuracy on the test set. From Figure 8, we can see that all the models make many mistakes, not only on the underrepresented classes. We notice that the convolutional LSTM layers struggle both in the case of the single-layer and two-layer encoder-decoder architecture. The multi-layer model shows only a marginally higher accuracy than the single-layer model, with the difference being negligible. On the other hand, the LSTM-based models show a substantial difference between the single and multi-layer versions, with the multi-layer model being around 24% more accurate. This could indicate that the single-layer LSTM did not have sufficient capacity. Interestingly, the distribution of mistakes across all the models appears to be quite similar, with even the best-performing model following the same pattern. This suggests that a model like "LSTM_multilayer" isn't truly learning mathematical operations, but rather mapping the latent space to more appropriate solutions in a way that resembles "memorization" rather than performing arithmetic operations. It seems that none of these models are able to extract the underlying arithmetic principles from the input images and instead simply memorize them.

Table 3: Image-to-text Model Performance

Model	Loss	Accuracy
convolstm	1.6586	0.4193
lstm	2.6941	0.5646
convolstm_multilayer	1.6548	0.4293
lstm_multilayer	0.8355	0.7081

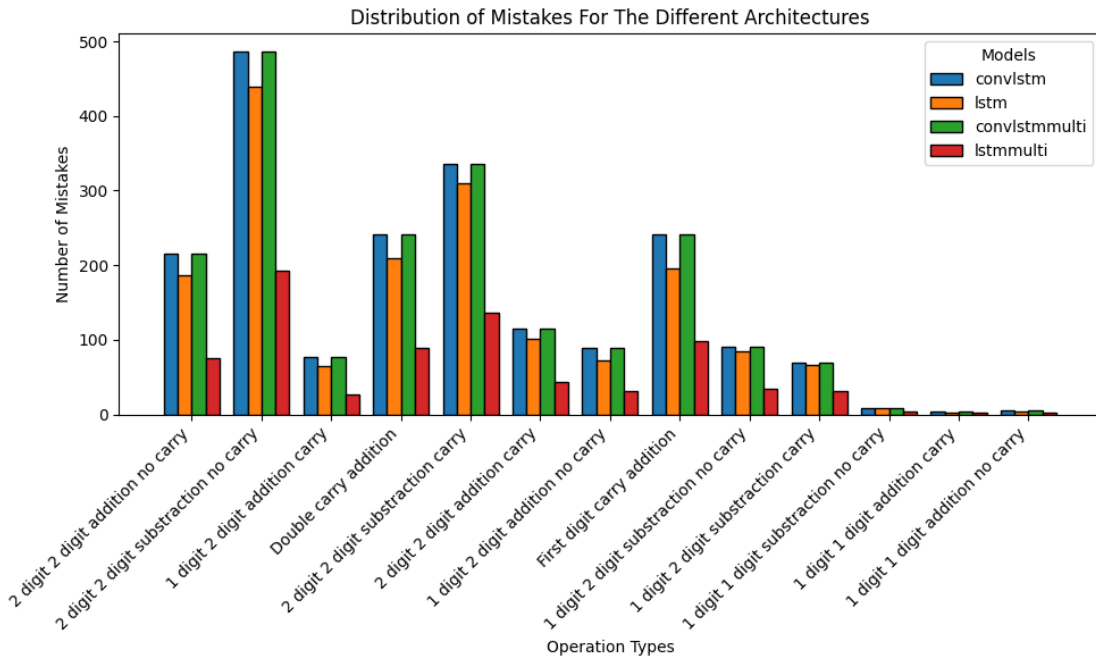


Figure 8: Mistake Distribution of Operation Types

2.4 Text-to-image RNN

The third subtask of Task 2 consists of a text-to-image recurrent neural network. The input data consists of the same one-hot encoded array (with dimensions 20,000x5x13) taken from the text-to-text model. The output array is a sequence of three 28x28-pixel images, each corresponding to a digit. The models are then trained to predict the arithmetic result and generate as output a sequence of three images. Each individual output of the model has dimensionality 3x28x28. The output is then assessed by determining the correctness and visual quality of the generated images by comparing it to the decoded input. Since the train-test split was experimented on in the section above, these models will only be run on the 90-10 split to obtain the best possible results.

The model architecture will be based on a encoder-decoder sequential architecture. The encoder will handle feature extraction from the input text, while the decoder will generate images based on the final output provided by the encoder. The encoder part of each of the fitted models will be sequential however the decoder was approached both in sequential and multi-headed output to see which architecture yielded the best results.

The main structure of the encoder for these models was constructed using LSTM(Long short term memory) sequential layers. By using LSTM we aim to train the encoder in such a way that it memorizes the potential arithmetic carry operation of each input sequence. Another advantage of LSTMs is lessening the effect of the vanishing gradient problem, which is usually amplified in simple recurrent networks. As shown in subtask 2.1 a single LSTM layer is enough to

generalize the hidden feature vector such that it retains enough information to achieve an accuracy of around 98%. However, the encoder structure and layers were also changed to allow for more flexibility.

For the decoder part, 3 main layers were used, simple **LSTM**, **2d transpose layers** and text **Convolutional LSTM layers**. The first approach is a **LSTM decoder**. By simply feeding 3 vectors from the decoder directly to a dense and reshape layer we can train the model directly to the image array. This model’s main advantage lies in its architectural simplicity and high parameter count. The second type of decoder trained is a **deconvolutional layer** decoder. By using a deconvolutional layer as output we aim to extract visual features from the hidden feature vector. This decoder leverages the ability of transposed convolutional layers (commonly referred to as deconvolutional layers) to generate visual features. The third type of decoder uses a special type of LSTM called a **convolutional LSTM**, the same layer used in section 2.2. With the convolutional LSTM as a decoder, we aim to transform a 3x28x28 feature map sequence into 3 distinct digits. The advantage of this compared to a deconvolutional layer would be the Long and short-term memory logic adapted from LSTMs to handle convolutional operations. In principle with a convolutional LSTM, we are trying to transform a "feature image" into a distinct digit image. These three decoders were chosen to evaluate different approaches to mapping the hidden feature vector to the output image array, balancing simplicity, feature extraction capabilities, and memory logic.

2.4.1 Approach

For this task, a total of 6 models were trained. These models vary greatly from one another not only in parameter counts but also in architecture. Each of the 6 models had to overcome different challenges during training with only 3 producing satisfactory results.

The main challenge of this task was getting the models to generate clear and readable images based on a hidden feature vector provided by the encoder. During training all of the models struggled to generate clear images. The third digit was especially difficult for the network to generalize to, due to the large variance compared with the middle and first digit. Thus the third image was usually very unclear and it often converged into a "grey blob" as shown in Figure 9.

This problem was fixed by increasing the original dataset by a factor of 10x. Note: since the augmentation is applied after the train-test data is split, only the train data is multiplied, while the test set remains untouched. By artificially increasing the train set size, we ensure that the model has the necessary data to generate distinct digits more effectively. This approach is similar to reducing the learning rate by helping the model learn more effectively, but it proved to be much more time-efficient.

In Table 4 each of the 6 defined model architectures are described. Once the input is processed sequentially by the encoder the hidden feature vector is fed to the decoder.

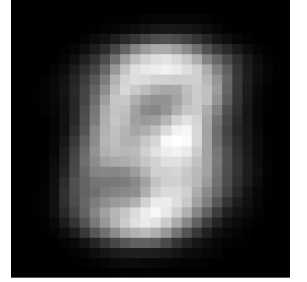


Figure 9: "Grey blob"

Table 4: Model architectures

Model Name	Encoder Layers	Decoder Layers	nr.params
Deconvo	LSTM, LSTM, LSTM RepeatVector	ConvTranspose, ConvTranspose, ConvTranspose	1,960,609
Separate	LSTM, LSTM	Dense, ConvTranspose, ConvTranspose, ConvTranspose	7,328,691
Vector	LSTM, LSTM, LSTM	LSTM, LSTM, LSTM, Dense, Dense	9,857,472
Decode.Encode.Deconvo	LSTM, LSTM RepeatVector	ConvTranspose, ConvLSTM	5,144,726
ConvLSTM	LSTM RepeatVector	ConvLSTM, ConvLSTM, ConvLSTM	2,764,456
Two_Part	LSTM, LSTM	Dense, ConvTranspose, ConvLSTM	553,693

As shown in the Table 4 above there are 4 types of models in total with one special case. The architecture of each decoder is constructed using the 3 types of layers(LSTM,Transpose,Convolutional LSTM), which were discussed in the previous section. The simpler model using only LSTM layers as decoders is represented by the "vector" model. This architecture decodes the image by transforming the hidden feature vector into a sequence of output vectors, each responsible for a single digit. These vectors are then reshaped and transformed into a 28x28 pixel grid of values.

A decoder constructed solely from deconvolutional layers was used in two models: "Deconvo" and "Separate." The "Deconvo" model simply repeats the output and decodes it three times, whereas the "Separate" model is more sophisticated, treating each digit as an individual output. This distinction in how the decoder is handled results in substantial differences in the final outcomes.

One model, "ConvoLSTM," was fitted using only convolutional LSTM layers to construct the decoder. Two other models, "Decode Encode Deconvolutional" and "Two Part," used a combination of both layer types. The "Two Part" model is a special case, as its encoder was first trained on a text-to-text task. Subsequently, a decoder was fitted, and the model was trained for image generation. The purpose of this model was to explore how the hidden feature vector impacts the quality of the generated images.

As seen in table 4 the parameter count varies quite a bit between the models. Due to time computational constraints further experimentation on these models was limited. The models with the most promising results during early testing which in our case were "Vector", "Separate" and "ConvoLSTM" have been further refined, thus the large discrepancy in parameter counts.

Each model was compiled with "binary cross-entropy" as a loss function and they were optimized using "Adam". Note: Due to the nature of this task the feedback metrics during training are difficult to interpret. Thus a random query was chosen and its prediction was displayed after each epoch to determine how well the training process was unfolding. The image in Figure 10 is the output of the model named "Vector", on the input sequence query: "7+72" during training for epochs: 1, 12, and 30. We can clearly see that through the epochs the image becomes sharper. This query is randomized from the data set, the image will be different on each individual training run.

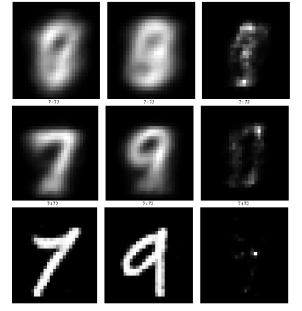


Figure 10: Training output prediction for epochs: 1, 12, 30

2.4.2 Results

The final results were obtained on the original test set. As mentioned above the test split is not augmented. Out of the test prediction results, 100 of them were randomly picked to be displayed and then the images were assessed by two criteria. First, are the images clear and or sharp enough to determine their value? Secondly, are the output values arithmetically correct? As mentioned earlier, before augmenting the training data, even the better-performing models struggled to converge, particularly when generating the third digit. However after the models were then trained on the augmented data, the results changed substantially. Table 11 has a sample from the prediction on the query: "27+82" for each of the models.

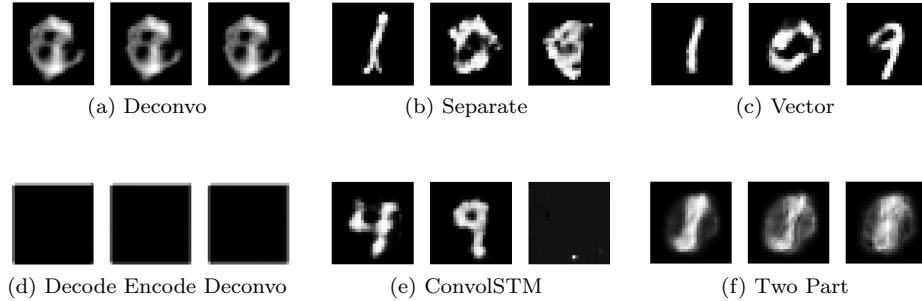


Figure 11: Prediction of each model on test query: "27+82"

Firstly assessing the clarity of the models is a straightforward process. Looking at 100 values from the output we see that only 3 models managed to get clear enough images such that numerical values can be derived. The simple "vector" model has the highest image fidelity as seen in Figure 11. The "Separate" model successfully displays a diverse range of digits but struggles with reconstructing some of them. The "ConvoLSTM" model seems to also be effectively generating clear and readable numbers. Of note are the models "Decode Encode Deconvo" and "Deconvo". Both of these models have failed to display any meaningful output. "Deconvo" seems to display an averaged image over the three separate digits, while "Decode Encode Deconvo" provides empty images for each value. This seems to be happening due to information loss between the transpose and convolutional layers although the exact reason could not be pinpointed within the computational constraints. Since no readable values can be obtained, these two models will not be further assessed for arithmetic correctness.

Examining the output of the "Vector" model for 100 samples reveals that it performs largely correct. Surprisingly, the model handles carry operations and negative value calculations quite well. While some generated samples were not clear enough to determine their values, these instances appear to be exceptions rather than the norm. The "Separate" model, on the other hand, struggled to produce highly distinct results, making it difficult to identify errors. However, in the clearer cases, it generally achieves accuracy, with only small deviations in digit values. The "ConvoLSTM" model presents an interesting case: it generates relatively clear images, yet none of its outputs were valid arithmetic predictions.

After evaluating the outputs from all the models, the "Vector" model emerges as the best-performing one. It consistently generates clear and sharp images and provides largely valid solutions for each query.

2.4.3 Discussions

Looking back at the results of all models, it is clear that "Vector" has the most appropriate architecture for this task. It is not only simple but, due to the additional time complexity introduced by convolutional operations in the other models, it is also the most easily scalable. The images generated by "Vector" are sharp and clear. Surprisingly, it handles arithmetic operations quite well. This may be due to the sequential output preserving "mathematical" memory, allowing the decoder to generate each digit appropriately.

As seen in the results, a decoder constructed with deconvolutional layers could potentially generate valid images. However, it is limited by its inability to handle sequential data, meaning such a decoder cannot output a sequence directly but rather three distinct values. As a result, the digits generated have no influence on one another, which can lead to invalid results. The layers should also be separated to avoid averaged outcomes, as observed in the "Deconvo" model. This more complex relationship is also evident when deconvolutional layers interact with convolutional LSTM layers; an inappropriate structure can lead to invalid results, as seen in the "Decode Encode Deconvo" model. "ConvoLSTM" manages to generate clear images; however, it seems to struggle with producing valid arithmetic results, even with large parameter counts.

Interestingly, given its low parameter count, the "Two Part" model managed to output the outline of distinct digits. This output is similar to earlier iterations of our current best-performing model, namely "Vector". However, the architecture limits the scalability of this model. Any changes to the encoder require training two separate models, which substantially increases the time complexity. As a result, experimentation with this model was limited to only a few iterations with low parameter counts.

2.5 Final Summary and Conclusions

The main goal of subtask 1 was to determine the most effective train-test split. After experimenting with four different splits, we observed that larger train sets consistently yielded significantly better performance. However, we also found that certain operation types remained underrepresented in these larger sets, causing the model to struggle with learning them. This outcome aligns with expectations, as smaller train sets lacked sufficient samples of these operation types, preventing the model from grasping their characteristics.

In the second subtask, we aimed to evaluate the performance of various models and determine how well they could learn mathematical operations from the input images. We found that the LSTM multi-layer model performed the best, achieving 77% accuracy on the test set. However, all models, including the best-performing one, made several mistakes, particularly on underrepresented classes. This outcome was expected, as the models struggled to extract the underlying arithmetic principles from the input images. Instead, they appeared to "memorize" solutions by mapping the latent space, rather than performing true arithmetic operations. Interestingly, the LSTM-based models showed a significant improvement with the multi-layer version, which was about 24% more accurate than the single-layer model, indicating that the single-layer LSTM lacked sufficient capacity. Overall, these results suggest that while multi-layer LSTM models outperformed their single-layer counterparts, none of the models were able to fully grasp the mathematical principles required for the task.

In the third subtask, which focused on image generation from text sequences, we found that the "Vector" model had the most appropriate architecture for the task. Its simplicity and scalability, coupled with sharp and clear image generation, made it the best performer. The sequential output of the "Vector" model preserved "mathematical" memory, aiding in accurate arithmetic operations. While deconvolutional layers in the "Deconvo" and "Decode Encode Deconvo" models showed potential, their inability to handle sequential data and the need for separated layers hindered performance. The "ConvoLSTM" model generated clear images but struggled with arithmetic, even with a large parameter count. Despite its low parameter count, the "Two Part" model managed to generate outlines of digits but lacked scalability, limiting further experimentation.

In conclusion, while the models can handle sequential data and generate valid results, further inspection reveals that it is highly challenging for them to learn actual arithmetic operations. Instead, they seem to converge and memorize solutions rather than genuinely understanding the underlying principles. This conclusion is evident throughout Task 2, from the data split in the first subtask, the distribution of mistakes in the second, to the augmentation required in the third subtask. Ultimately, the models rely more on memorization and brute-force solutions than on truly learning the arithmetic operations.

References

- [1] Simon J.D. Prince. *Understanding Deep Learning*. The MIT Press, 2023. URL: <http://udlbook.com>.
- [2] TensorFlow. *Convolutional Variational Autoencoder*. Mar. 2021. URL: <https://www.tensorflow.org/tutorials/generative/cvae> (visited on 12/10/2024).
- [3] TensorFlow. *Deep Convolutional Generative Adversarial Network*. Mar. 2021. URL: <https://www.tensorflow.org/tutorials/generative/dcgan> (visited on 12/10/2024).